

Inférence Probabiliste et Estimation d'État

INF8422 : Perception Robotique et Intelligence Spatiale

Prof. Pierre-Yves Lajoie

**POLYTECHNIQUE
MONTREAL**

Agenda

1. Robotique Probabiliste
2. Rappels de Probabilités
3. Estimation d'État : MLE & MAP
4. Graphes de Facteurs
5. Optimisation Non-Linéaire
6. Groupes de Lie
7. Résumé – Le Flow d'Estimation d'État Complet

Ce cours

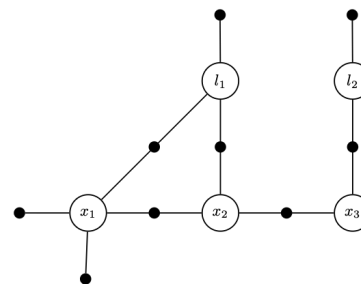
Au dernier cours, vous avez vu le SLAM en action :

- **Feature extraction, RANSAC, PnP** → contraintes géométriques
- **Bundle Adjustment** → minimisation de l'erreur de reprojection
- **Graphe de facteurs VIO** → structure de l'optimisation

Des termes n'ont pas été bien définis :

"Minimiser l'erreur" – pourquoi moindres carrés ? Quelle erreur au sens probabiliste ?

"Résoudre le graphe" – comment ? Quel algorithme ?



Le graphe de facteurs SLAM – ce que nous allons formaliser

Ce cours répond à ces questions

Probabilités → Bayes → MLE/MAP → Moindres Carrés → Graphes de Facteurs → Gauss-Newton → Lie Groups

Les fondements du SLAM Back-end

Robotique Probabiliste

Le modèle de mesure

En robotique probabiliste, une mesure est une variable aléatoire.

$$z_t = h(x_t) + \epsilon_t$$

- z_t : Le vecteur de mesure à l'instant t .
- x_t : L'état du robot/monde (ex : position, carte).
- $h(\cdot)$: La **fonction d'observation** (modèle physique du capteur).
- ϵ_t : Le bruit de mesure (souvent Gaussien $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$).

Perception

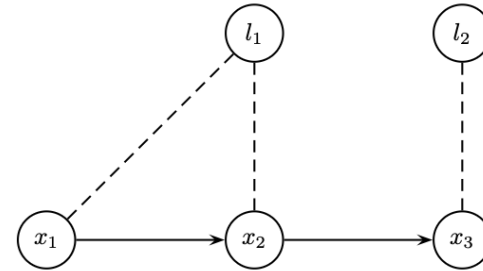
Le but de la perception est d'inverser h pour trouver x sachant z .

L'Incertainitude en Robotique

Contrairement à une simulation parfaite, le monde réel est **incertain**.

Sources d'incertitude :

- **Capteurs** : Les mesures sont bruitées. $z \neq h(x)$, soit $\|z - h(x)\|^2 > 0$.
- **Actionneurs** : Les roues glissent, les moteurs ne sont pas parfaits. $u \neq \Delta x$.
- **Modèles** : Nos équations sont des simplifications de la physique réelle.
- **Environnement** : Des objets bougent, la luminosité change.



Exemple SLAM : poses et landmarks inconnus

Conséquence fondamentale

On ne peut jamais connaître l'état du robot avec une précision parfaite. Il faut **quantifier et propager** cette incertitude.

La Robotique Probabiliste

"Probabilistic robotics is a new approach to robotics that pays tribute to the uncertainty inherent in robot perception and action." – Sebastian Thrun, 2005

Idée clé :

Au lieu de dire "Le robot est à la position (x, y) ", on dit :

"La position du robot suit une distribution de probabilité $\mathcal{N}(\mu, \Sigma)$ "

⇒ États → Distributions de probabilité

Ce que cela change

- Un **état** est une **densité de probabilité** $p(x)$.
- Une **mesure** met à jour cette densité (règle de Bayes).
- Une **action** propage l'incertitude (propagation d'erreur).

En SLAM

On cherche à estimer la densité conditionnelle :

$$P(X, M \mid Z)$$

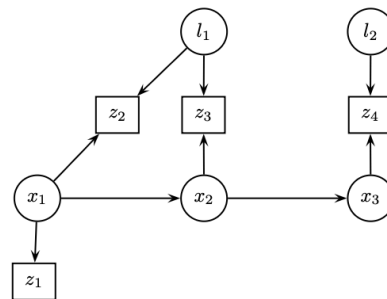
- X : trajectoire du robot, M : carte, Z : mesures capteurs.

Modélisation Probabiliste SLAM

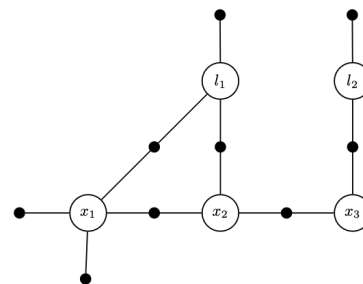
Scénario : Trajectoire à 3 poses x_1, x_2, x_3 et deux landmarks l_1, l_2 .

Densité jointe (via réseau Bayésien) :

$$P(X, Z) = p(x_1) p(x_2|x_1) p(x_3|x_2) p(l_1) p(l_2) \\ \cdot p(z_1|x_1) p(z_2|x_1, l_1) p(z_3|x_2, l_1) p(z_4|x_3, l_2)$$



Réseau Bayésien



Graphe de facteurs équivalent

Graphes de facteurs

Mieux adaptés pour l'inférence $P(X|Z)$ que les réseaux Bayésiens. On remplace les densités conditionnelles par des **facteurs** :

$$P(X | Z) \propto \prod_i \phi_i(X_i)$$

Rappels de Probabilités

Variables Aléatoires

Cas discret – PMF

Une variable X prend des valeurs dans $\{x_1, \dots, x_n\}$.

$$P(X = x_i) = p_i, \quad \sum_{i=1}^n p_i = 1$$

Classification sémantique

Une caméra voit un objet. Qu'est-ce que c'est ?

- $P(X = \text{Vélo}) = 0.1$
- $P(X = \text{Piéton}) = 0.8$
- $P(X = \text{Voiture}) = 0.1$

Cas continu – PDF

La fonction de densité $p(x)$ satisfait :

$$\int_{-\infty}^{\infty} p(x) dx = 1$$

Densité $\neq$ Probabilité

- $P(X = x) = 0$.
- On calcule : $P(a \leq X \leq b) = \int_a^b p(x) dx$.

Indépendance

Si X et Y sont indépendants :

$$p(x, y) = p(x) \cdot p(y)$$

Règles de Probabilités

Règle de la Somme (Marginalisation) :

$$p(x) = \int p(x, y) dy$$

Intuition : "écraser" la distribution 2D sur l'axe x .

Règle du Produit :

$$p(x, y) = p(x | y) p(y) = p(y | x) p(x)$$

Règle de la Chaîne (Chain Rule) :

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$$

Hypothèse de Markov :

Si x_{t+1} ne dépend que de x_t (pas de l'historique) :

$$p(x_0, x_1, x_2) = p(x_0) \cdot p(x_1 | x_0) \cdot p(x_2 | x_1)$$

Indépendance Conditionnelle

X et Y sont indépendants sachant Z :

$$p(x, y | z) = p(x | z) p(y | z)$$

Théorème de Bayes en Robotique

$$p(x | z) = \frac{p(z | x) p(x)}{p(z)}$$

- $p(x)$: **Prior** – croyance sur l'état avant la mesure.
- $p(z | x)$: **Likelihood** – vraisemblance de la mesure sachant l'état.
- $p(x | z)$: **Posterior** – croyance sur l'état sachant la mesure.
- $p(z)$: **Evidence** – normalisation.

Capteur de porte

État : porte Ouverte (O) ou Fermée (F). Prior :
 $P(O) = P(F) = 0.5$.

Capteur bruité : $P(z_O | O) = 0.8, P(z_O | F) = 0.4$.

Si le capteur dit "Ouvert" :

$$P(z_O) = 0.8 \times 0.5 + 0.4 \times 0.5 = 0.6$$

$$P(O | z_O) = \frac{0.8 \times 0.5}{0.6} \approx \mathbf{0.67}$$

Même avec une mesure "Ouvert", il reste 33% de chances que la porte soit fermée !

La Loi Normale (Gaussienne)

Le modèle de bruit standard en robotique – justifié par le **Théorème Central Limite**.

Univariée

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

- μ : Moyenne (centre).
- σ^2 : Variance (incertitude).

Règle 68-95-99.7 :

- $\mu \pm 1\sigma \rightarrow 68\%$ de probabilité.
- $\mu \pm 2\sigma \rightarrow 95\%$.
- $\mu \pm 3\sigma \rightarrow 99.7\%$.

Multivariée ($x \in \mathbb{R}^n$)

$$p(x) = \det(2\pi\Sigma)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

- $\mu \in \mathbb{R}^n$: Vecteur moyenne.
- $\Sigma \in \mathbb{R}^{n \times n}$: Matrice de covariance.

Distance de Mahalanobis

$$D_M^2 = (x - \mu)^T \Sigma^{-1}(x - \mu)$$

Distance euclidienne pondérée par l'incertitude.

Propriétés des Gaussiennes

Propriété 1 – Transformation Linéaire

Soit $x \sim \mathcal{N}(\mu, \Sigma)$ et $y = Ax + b$.

$$y \sim \mathcal{N}(A\mu + b, A\Sigma A^T)$$

Exemple : changement de repère caméra $\rightarrow$ monde.

$$A = R_C^W.$$

$$\Sigma^W = R_C^W \Sigma^C R_W^C$$

Propriété 2 – Fusion (Produit)

Le produit de deux Gaussiennes est une Gaussienne :

$$\mathcal{N}(\mu_{new}, \Sigma_{new}) \propto \mathcal{N}(\mu_1, \Sigma_1) \cdot \mathcal{N}(\mu_2, \Sigma_2)$$

Les **matrices d'information** s'additionnent :

$$\Sigma_{new}^{-1} = \Sigma_1^{-1} + \Sigma_2^{-1}$$

Intuition

Plus on accumule de mesures, plus la covariance Σ diminue. L'incertitude est déterminée par les données.

Estimation d'État : MLE & MAP

Le Problème d'Estimation

Données :

- $Z = \{z_1, \dots, z_m\}$: Mesures bruitées.
- $u = \{u_1, \dots, u_k\}$: Commandes de contrôle (optionnel).

Inconnue :

- x : L'état réel du système (pose du robot, carte, calibration...).

But : Trouver la "meilleure" estimation $\hat{x}$ qui explique les données.

Modèle de mesure :

$$z_i = h_i(x) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \Sigma_i)$$

La vraisemblance d'une mesure gaussienne :

$$p(z_i | x) \propto \exp\left(-\frac{1}{2}\|h_i(x) - z_i\|_{\Sigma_i}^2\right)$$

Indépendance conditionnelle :

$$p(z_{1:m} | x) = \prod_{i=1}^m p(z_i | x)$$

Deux approches

- **MLE** (Maximum Likelihood Estimation). - **MAP** (Maximum A Posteriori) : incorpore un prior $p(x)$.

Mise à jour bayésienne : une mesure à la fois

Position x fixe · $z_k = x + \varepsilon_k$ · bruits gaussiens indépendants · σ = écart-type

① Observer

prior + mesure

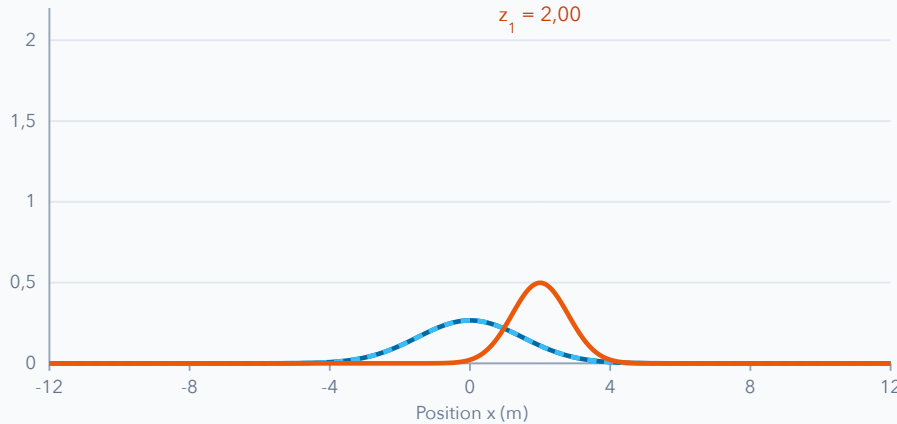
② Fusionner

produit normalisé

③ Retenir

posterior mémorisé

--- Prior initial p_0 fixe — Avant : p_0 — Vraisemblance — Après : p_1
Densité · échelles fixes



$$p_k(x) = \frac{p(z_k|x) p_{k-1}(x)}{\int p(z_k|u) p_{k-1}(u) du}$$

Avant z_1
 $\mu = 0,00 \cdot \sigma = 1,50$

Mesure z_1
 $z = 2,00 \cdot \sigma_z = 0,80$

1. Une mesure à examiner

L'orange montre quels x rendent z_1 plausible. La mesure n'est pas encore intégrée.

Au départ, p_0 est aussi le prior de cette mise à jour : les deux courbes bleues se superposent.

$$\frac{1}{\sigma_k^2} = \frac{1}{\sigma_{k-1}^2} + \frac{1}{\sigma_z^2}$$

Les précisions s'additionnent.

μ_0 0,00 σ_0 1,50 z_1 2,00 σ_z 0,80

Calculer le posterior →

► Lecture

↺ Réinitialiser

Régler la mesure, puis avancer.

Chaque cycle suppose une nouvelle observation indépendante, même si z est identique. Réutiliser la même observation compterait deux fois l'information.

MLE → Moindres Carrés

Maximum de Vraisemblance :

$$\hat{x}_{MLE} = \arg \max_x p(z_{1:m} | x) = \arg \max_x \prod_i p(z_i | x)$$

Astuce Log (log est monotone croissant) :

$$\begin{aligned} &= \arg \max_x \sum_i \ln p(z_i | x) \\ &= \arg \max_x \sum_i \left(-\frac{1}{2} \|h_i(x) - z_i\|_{\Sigma_i}^2 \right) \end{aligned}$$

Maximiser le négatif = Minimiser :

$$\hat{x}_{MLE} = \arg \min_x \sum_i \frac{1}{2} \|h_i(x) - z_i\|_{\Sigma_i}^2$$

Conclusion majeure

Sous hypothèse de bruit Gaussien, **Maximum de Vraisemblance = Moindres Carrés Pondérés.**

$$\|e\|_{\Sigma}^2 = e^T \Sigma^{-1} e$$

Capteur précis (σ petit) → poids $1/\sigma^2$ **grand**.

Capteur imprécis (σ grand) → poids **petit**.

MAP → Moindres Carrés avec Prior

Maximum A Posteriori :

$$\hat{x}_{MAP} = \arg \max_x p(x \mid Z) \propto p(Z \mid x) p(x)$$

Si le prior est Gaussien : $x \sim \mathcal{N}(x_0, \Sigma_0)$, le problème devient :

$$\hat{x}_{MAP} = \arg \min_x \left[\sum_i \|h_i(x) - z_i\|_{\Sigma_i}^2 + \|x - x_0\|_{\Sigma_0}^2 \right]$$

Le prior agit comme une **mesure additionnelle** qui tire x vers x_0 .

Exemple 1D – Odométrie

Trajectoire de robot à 3 positions x_0, x_1, x_2 . Mesures de déplacement : $d_1 = 2$ m, $d_2 = 1.5$ m ($\sigma_d = 0.5$).

Prior : $x_0 \approx 10$ m ($\sigma_0 = 0.1$).

$$E = \underbrace{\frac{(x_0 - 10)^2}{0.1^2}}_{\text{prior/ancrage}} + \underbrace{\frac{(x_1 - x_0 - 2)^2}{0.5^2}}_{\text{Mouv. 1}} + \underbrace{\frac{(x_2 - x_1 - 1.5)^2}{0.5^2}}_{\text{Mouv. 2}}$$

Solution : $\hat{x}_0 \approx 10, \hat{x}_1 \approx 12, \hat{x}_2 \approx 13.5$.

Sans prior : système flottant

Sans ancrage, si (x_0, x_1, x_2) est une solution, $(x_0 + c, x_1 + c, x_2 + c)$ l'est aussi. Le système est sous défini, il y a une infinité de solutions.

Intuition : Analogie des Ressorts

Chaque terme de la fonction de coût agit comme un **ressort** qui tire l'état x vers la valeur observée z_i .

$$E(x) = \sum_i \underbrace{\frac{1}{2} \|h_i(x) - z_i\|_{\Sigma_i}^2}_{\text{énergie du ressort } i}$$

- Énergie du ressort : $E = \frac{1}{2} k \Delta x^2$.
- Raideur : $k = \Sigma_i^{-1}$ (inverse de l'incertitude/covariance).
- Allongement : $\Delta x = h(x) - z$ (erreur de mesure).

Minimiser $E \iff$

Trouver l'équilibre mécanique

Pourquoi les poids $1/\sigma^2$?

Toutes les mesures ne naissent pas égales :

$$E(x) = \frac{(h_1(x) - z_1)^2}{\sigma_1^2} + \frac{(h_2(x) - z_2)^2}{\sigma_2^2} + \dots$$

- Capteur **précis** (σ petit) $\rightarrow$ ressort **raide** $\rightarrow$ poids **grand**.
- Capteur **imprécis** (σ grand) $\rightarrow$ ressort **mou** $\rightarrow$ poids **petit**.

L'algorithme écoute davantage les capteurs précis.

Ressorts : variance et fermeture de boucle

Chaîne

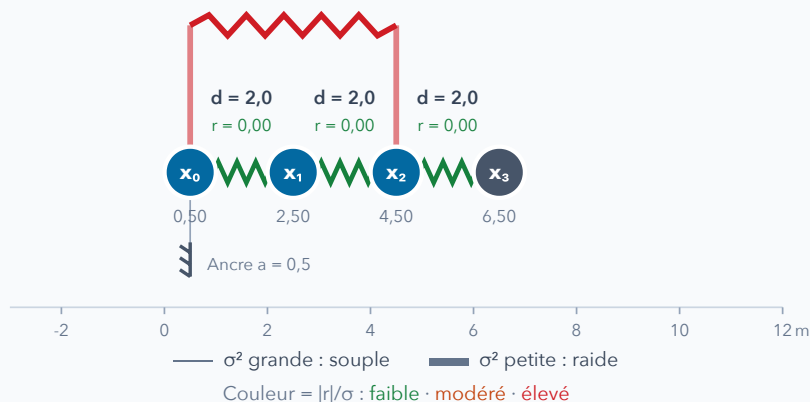
Boucle partielle : $x_0 \leftrightarrow x_2$

Positions en 1D · glisser les nœuds, puis relaxer

Boucle : $x_2 - x_0 \approx 3,0$ m

$r_{02} = 1,00$ m

x_3 hors de la boucle



Coût J **5,000**

Minimum J* **0,833**

À relaxer

$$J(x) = \frac{1}{2} \sum_i \frac{r_i^2}{\sigma_i^2}, \quad k_i = \frac{1}{\sigma_i^2}$$

Contrainte	Cible (m)	Variance σ^2 (m ²)
Prior sur x_0 $k = 10,0$	0,5	0,10
$x_1 - x_0$ $k = 4,0$	2,0	0,25
$x_2 - x_1$ $k = 4,0$	2,0	0,25
$x_3 - x_2$ $k = 4,0$	2,0	0,25
Boucle $x_2 - x_0$ $k = 10,0$	3,0	0,10

Prior : $r_0 = x_0 - a$ · Lien : $r_{ij} = x_j - x_i - d_{ij}$

Conflit : le chemin $0 \rightarrow 1 \rightarrow 2$ demande 4,0 m, la boucle 3,0 m. Les ressorts les moins précis absorbent davantage l'écart.

À l'équilibre, les forces se compensent ; les ressorts peuvent rester tendus. x_3 suit x_2 et conserve sa distance cible.

Relaxer → équilibre

Il Pause

Perturber les positions

↺ Réinitialiser

Animation : Newton amorti · modifier σ^2 , puis relaxer

Robustesse aux Aberrants – Faiblesse des Moindres Carrés

Les moindres carrés minimisent $\sum_i \frac{1}{2} \|r_i\|^2$: le coût croît en r^2 . Une seule mesure aberrante (erreur énorme) **domine** la somme et **tire** toute la solution.

Mesures aberrantes

L'**aliasing perceptuel** génère de **fausses fermetures de boucle**. Injectées telles quelles dans un back-end L_2 , elles **effondrent** la carte. Il faut rendre l'optimisation robuste.

Réthodes avancées pour la robustesse

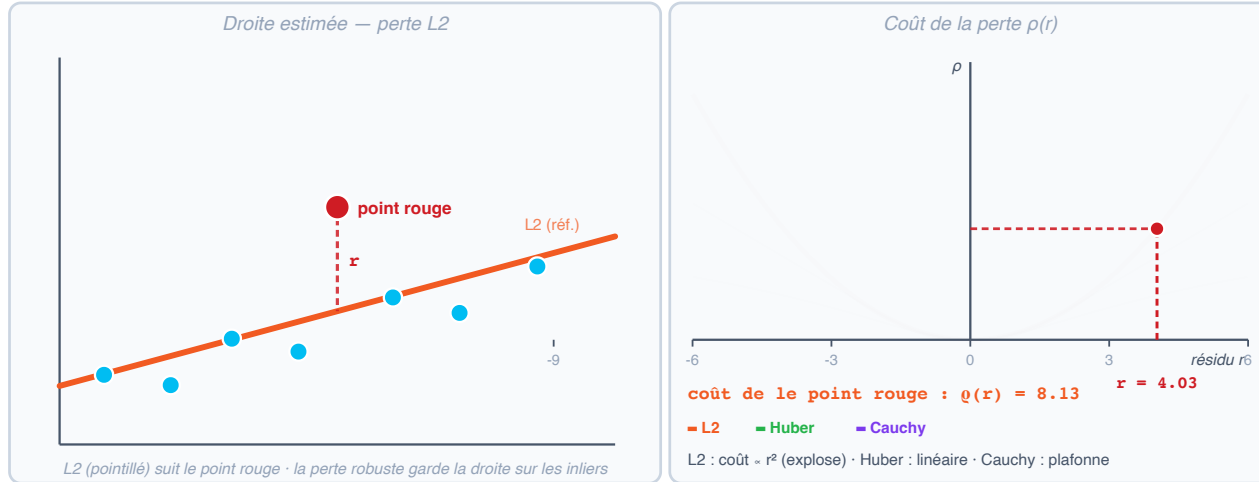
Exemples de méthodes avancées: *GNC* (Yang et al. 2020) varie progressivement la robustesse. *PCM* (Mangelson et al. 2018) filtre géométriquement les mesures incohérentes.

Noyaux robustes $\rho(r)$ – on remplace $\frac{1}{2}r^2$ par une fonction qui **plafonne** :

$$\hat{X} = \arg \min_X \sum_i \rho(\|r_i\|_{\Sigma_i})$$

- **Huber** : quadratique près de 0, **linéaire** au-delà de δ .
- **Cauchy / Geman-McClure** : redescendante – les gros résidus sont **quasi ignorés**.

Estimation Robuste – Visualisation Interactive



L2 Huber Cauchy y du point rouge 9.2

Éloignez le point rouge : sous **L2** la droite le suit ; sous **Huber/Cauchy** son coût plafonne et la droite tient.

9

Graphes de Facteurs

Du SLAM au Graphe de Facteurs

Nous avons vu le graphe de facteurs pour le VIO :

Poses $T_i \xrightarrow{[\text{IMU}]} T_{i+1}$, L_j vu depuis plusieurs T_i

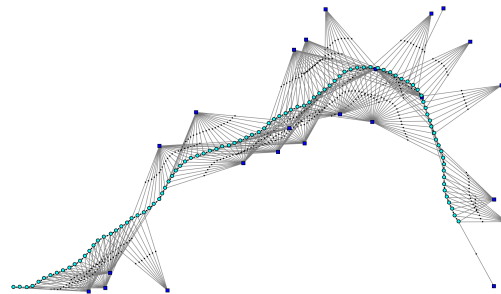
Maintenant, posons la **définition formelle**.

Un graphe de facteurs représente la **factorisation de la probabilité jointe** :

$$P(X \mid Z) \propto \prod_k \phi_k(X_k)$$

où chaque **facteur** ϕ_k encode une mesure (ou un prior) comme une vraisemblance gaussienne :

$$\phi_k(X_k) \propto \exp\left(-\frac{1}{2}\|h_k(X_k) - z_k\|_{\Sigma_k}^2\right)$$



Graphe de facteurs du SLAM

Connexion au cours précédent

Le Bundle Adjustment minimise $\sum \|z_{ij} - h(T_j, P_i)\|^2$, c'est exactement la MAP sur ce graphe de facteurs, avec chaque reprojection comme un facteur.

Graphes de Facteurs – Définition

Un graphe de facteurs représente la **factorisation de la probabilité jointe** :

$$P(X | Z) \propto \prod_i \phi_i(X_i)$$

Éléments :

- **Nœuds (variables)** : États inconnus X_i (poses x_k , landmarks l_j , calibration...).
- **Facteurs** ϕ_i : Fonctions de potentiel associées aux mesures Z et aux priors.

Les mesures observées sont **incorporées comme paramètres** des facteurs, pas comme nœuds.

Avantages par rapport aux réseaux Bayésiens :

- Structure visuelle claire pour l'inférence.
- Directement lié à la structure creuse du problème d'optimisation.
- Facilement implémentable (GTSAM, g2o, Ceres).

Exemple: Facteur odométrique

$$\phi(x_i, x_{i+1}) \propto \exp\left(-\frac{1}{2}\|x_{i+1} \ominus x_i - \Delta u_i\|_{\Sigma}^2\right)$$

Connecte deux poses consécutives avec la mesure d'odométrie Δu_i .

MAP sur les Graphes de Facteurs

Inférence MAP = maximiser le produit de tous les facteurs :

$$X^{MAP} = \arg \max_X \prod_i \phi_i(X_i)$$

Avec un modèle de bruit gaussien pour chaque facteur :

$$\phi_i(X_i) \propto \exp\left(-\frac{1}{2}\|h_i(X_i) - z_i\|_{\Sigma_i}^2\right)$$

Le **negative log** transforme le problème en **moindres carrés non-linéaires** :

$$X^{MAP} = \arg \min_X \sum_i \|h_i(X_i) - z_i\|_{\Sigma_i}^2$$

Fusion de capteurs naturelle

Minimiser la fonction objectif $\sum_i \|h_i(X_i) - z_i\|_{\Sigma_i}^2$
combine automatiquement toutes les sources de mesure :

- Odométrie (roues, IMU)
- Observations visuelles (features, reprojection)
- LiDAR, GPS, loop closures...

Chaque facteur contribue proportionnellement à sa précision Σ_i^{-1} .

Pourquoi Linéariser ? Le Problème est Non-Linéaire

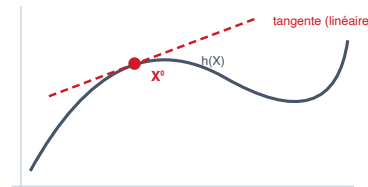
La MAP nous a donné un objectif de **moindres carrés** :

$$\hat{X} = \arg \min_X \sum_i \|h_i(X) - z_i\|_{\Sigma_i}^2$$

Les h_i sont souvent **non-linéaires** (projection, IMU...). En SLAM, on ne dispose généralement pas d'une solution analytique directe. Gauss-Newton et LM résolvent une succession de **systèmes linéaires**.

La stratégie

Linéariser h autour de X^0 (Taylor 1er ordre), résoudre les **moindres carrés linéarisés**, mettre à jour X , puis **itérer**.



Plan de la section (Lift-Solve-Retract)

1. **Lift** : linéariser $\rightarrow$ Jacobien, whitening, forme matricielle.
2. **Solve** : équations normales + Cholesky (creux).
3. **Retract + itérer** : Gauss-Newton / LM (section suivante).

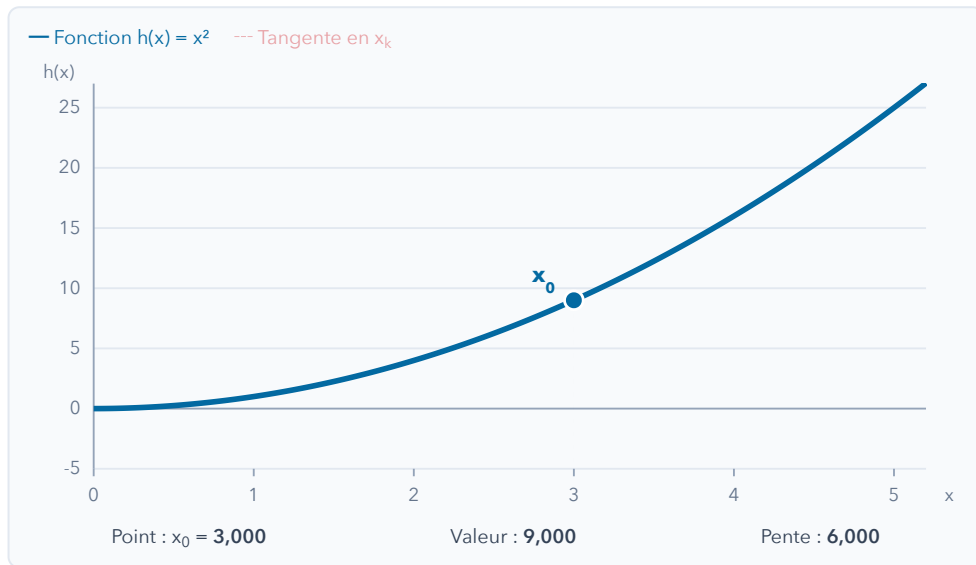
Rappel de Taylor : une approximation locale

① Choisir un point

② Tracer la tangente

③ Voir l'erreur

④ Calculer un pas



Point x_k 3,000 Déplacement Δx 0,750

Étape suivante →

► Démonstration

↺ Réinitialiser

$$h(x_k + \Delta x) \approx h(x_k) + h'(x_k) \Delta x$$

1. Partir de ce qu'on connaît

On choisit un point x_k sur la courbe. Ici, $h(x) = x^2$ et $h'(x) = 2x$.

$$h(x_k) = 9,00$$

$$h'(x_k) = 6,00$$

La valeur et la pente à ce point vont définir notre approximation locale.

En plusieurs dimensions, la pente devient le Jacobien :

$$h(\mathbf{x} + \Delta \mathbf{x}) \approx h(\mathbf{x}) + J(\mathbf{x}) \Delta \mathbf{x}$$

Le point de développement reste fixe pendant le pas.

Taylor rend le prochain calcul plus simple. Il fournit une approximation locale ; la fonction d'origine reste non linéaire.

Linéarisation : le Jacobien

Développement de Taylor au 1er ordre de h autour de X^0 , pour une perturbation Δ :

$$h(X^0 + \Delta) \approx h(X^0) + J \Delta$$

Le Jacobien J

$$J = \left. \frac{\partial h}{\partial X} \right|_{X^0}, \quad J_{ij} = \frac{\partial h_i}{\partial X_j}$$

Matrice des **dérivées partielles** :

- **ligne i** = variation du résidu i ,
- **colonne j** = par rapport à la variable X_j .

SLAM 1D

Facteur d'odométrie $r = (x_1 - x_0) - 2$. Ses dérivées :

$$\frac{\partial r}{\partial x_0} = -1, \quad \frac{\partial r}{\partial x_1} = +1$$

→ ligne du Jacobien $[-1, 1]$.

Ici h est **linéaire** $\Rightarrow J$ est **constant**.

On cherche la correction Δ

À cette itération, $X^0 \in \mathbb{R}^n$ est **fixé**. On cherche le déplacement $\Delta = X - X^0$.

1. Changement de variable

Le problème initial est :

$$\hat{X} = \arg \min_X \sum_i \|h_i(X) - z_i\|_{\Sigma_i}^2.$$

Remplacer X par $X^0 + \Delta$ donne :

$$\Delta_{\text{exact}}^* = \arg \min_{\Delta} \sum_i \|h_i(X^0 + \Delta) - z_i\|_{\Sigma_i}^2.$$

2. Linéariser (approximation locale)

Au point fixé X^0 , Taylor donne :

$$\begin{aligned} h_i(X^0 + \Delta) - z_i &\approx h_i(X^0) + J_i \Delta - z_i \\ &= J_i \Delta - (z_i - h_i(X^0)). \end{aligned}$$

On minimise alors ce **modèle approché** :

$$\Delta^* = \arg \min_{\Delta} \sum_i \|J_i \Delta - (z_i - h_i(X^0))\|_{\Sigma_i}^2$$

J_i et $z_i - h_i(X^0)$ sont **constants** pendant cette résolution ; seule Δ varie.

Whitening (Blanchiment du bruit) ?

Chaque mesure peut aussi avoir une *incertitude différente*. On ne peut pas les additionner naïvement dans une somme de carrés.

La **distance de Mahalanobis** pondère chaque erreur par son incertitude :

$$\|e\|_{\Sigma}^2 = e^{\top} \Sigma^{-1} e$$

Un capteur précis (σ petit) pèse **plus lourd** (poids Σ^{-1}).

Le blanchiment

On factorise $\Sigma^{-1} = \Sigma^{-\top/2} \Sigma^{-1/2}$ et on **change de variable** :

$$e' = \Sigma^{-1/2} e \Rightarrow \|e'\|^2 = \|e\|_{\Sigma}^2$$

Après blanchiment, tous les résidus ont un bruit **unité** $\mathcal{N}(0, I)$, directement **comparables**.

$$A_i = \Sigma_i^{-1/2} J_i, \quad b_i = \Sigma_i^{-1/2} (z_i - h_i(X^0))$$

Du Moindres Carrés à la Forme Matricielle

$$A_i = \Sigma_i^{-1/2} J_i, \quad b_i = \Sigma_i^{-1/2} (z_i - h_i(X^0))$$

On **empile** tous les facteurs blanchis. 1 ligne par résidu :

$$A = \begin{bmatrix} A_1 \\ \vdots \\ A_m \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix}$$

Le problème devient un **moindres carrés linéaire** :

$$\Delta^* = \arg \min_{\Delta} \|A\Delta - b\|^2$$

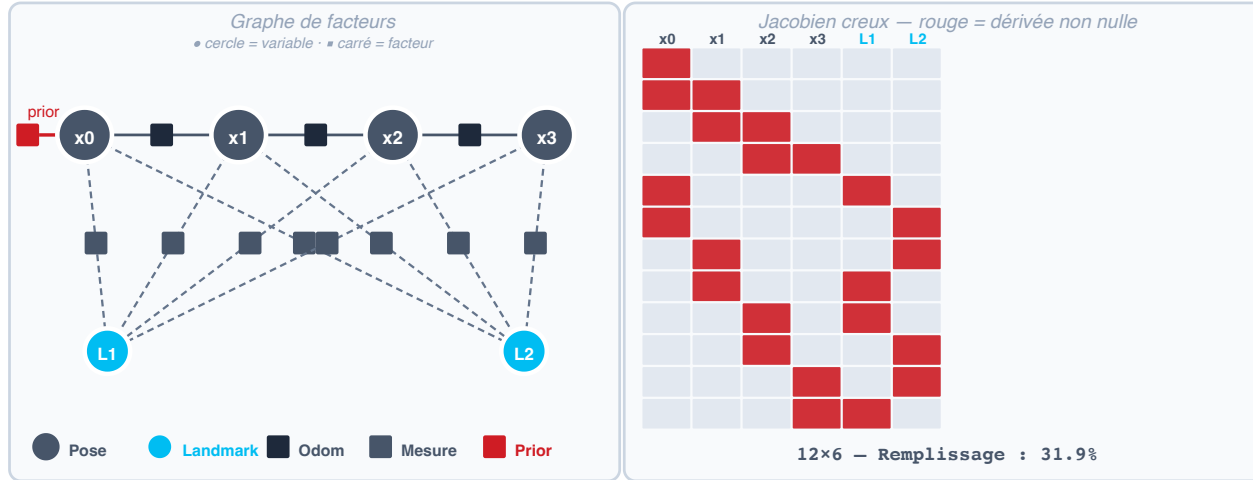
- **1 ligne** = 1 facteur · **1 colonne** = 1 degré de liberté.
- Chaque ligne ne touche que **quelques colonnes** $\Rightarrow A$ est dite **creuse** (sparse).

Notre SLAM 1D en boucle (4 poses)

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \\ -1 & 0 & 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 2 \\ 2 \\ 1 \\ 5 \end{bmatrix}$$

5 facteurs : ancre x_0 , 3 odométries, 1 **fermeture de boucle** $x_3 - x_0$ (dernière ligne).

Graphe de Facteurs et Jacobien Creux



Poses 4 Landmarks 2

12 facteurs × 6 variables — Le graphe grandit mais la Jacobienne reste creuse !

Dérivation des Équations Normales

On minimise $F(\Delta) = \frac{1}{2} \|A\Delta - b\|^2$
 $= \frac{1}{2} (A\Delta - b)^\top (A\Delta - b).$

Gradient (dérivation matricielle) :

$$\nabla_{\Delta} F = A^\top (A\Delta - b)$$

Condition d'optimalité $\nabla_{\Delta} F = 0$:

$$\boxed{A^\top A \Delta^* = A^\top b}$$

Propriétés de $\Lambda = A^\top A$

- **Symétrique** : $\Lambda^\top = \Lambda.$
- **Définie positive** (donc inversible) si A est de **rang colonne plein**, problème bien contraint.

Notre SLAM 1D

$$\Lambda = \begin{bmatrix} 3 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{bmatrix}, \quad A^\top b = \begin{bmatrix} -7 \\ 0 \\ 1 \\ 6 \end{bmatrix}$$

Rappel : Résoudre un Système Linéaire

Problèmes où on cherche Δ tel que $W\Delta \approx v$.

Deux cas:

- **W carrée inversible** : solution unique $\Delta = W^{-1}v$, mais on **n'inverse jamais** explicitement (coûteux et instable).
- **W rectangulaire**, plus de mesures que d'inconnues ($m > n$) : système **surdéterminé** $\Rightarrow$ pas de solution unique $\Rightarrow$ **moindres carrés**.

Familles de méthodes

- **Directes** : élimination de Gauss, **Cholesky**, QR
– nombre fini d'opérations, solution exacte (aux arrondis près).
- **Itératives** : gradient conjugué (CG)... pour les très gros systèmes.

Ce cours : **Cholesky creuse** sur les équations normales.

Ne jamais inverser

Calculer Λ^{-1} est $O(n^3)$ et détruit la sparsité.

À la place, on **factorise** puis on **substitue**.

Élimination, Ordering et Fill-in

Équations normales:

$$\Lambda = A^\top A$$

$$\boxed{\Lambda \Delta = A^\top b}$$

Factoriser Λ = **éliminer** les variables une à une (élimination de Gauss / Cholesky).

Éliminer une variable **connecte entre eux tous ses voisins** dans le graphe → de **nouvelles** arêtes = de **nouveaux non-zéros** dans Λ : le **fill-in**.

Plus de fill-in $\Rightarrow \Lambda$ moins creuse $\Rightarrow$ factorisation plus coûteuse.

L'ordre compte énormément

L'**ordre d'élimination** détermine le fill-in :

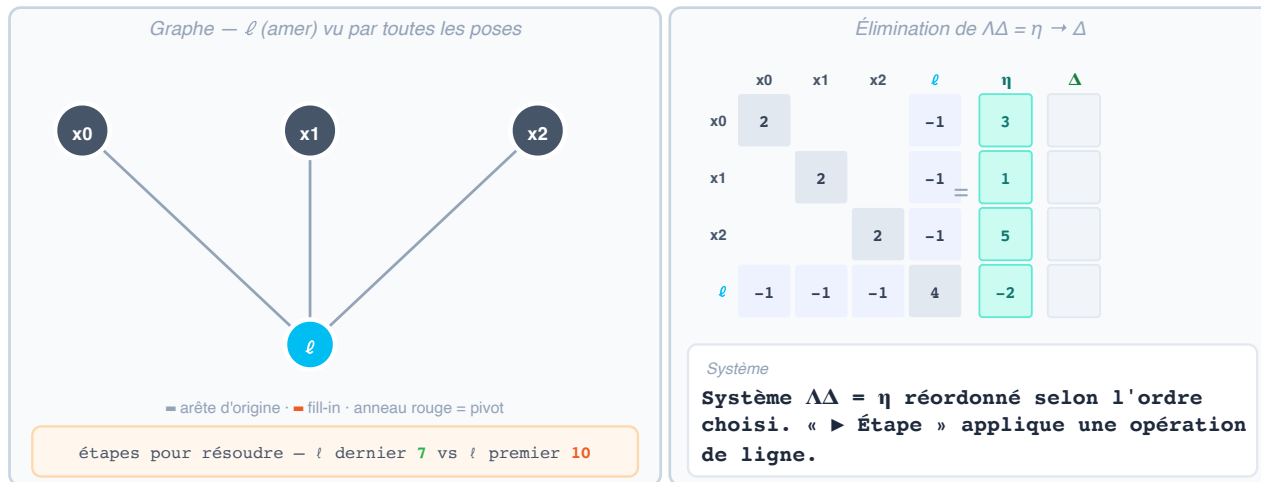
- **amer** vu par toutes les poses, éliminé **en dernier** → **0 fill-in**.
- le **même** éliminé **en premier** → relie toutes les poses → fill-in **massif**.

En pratique

Des heuristiques (**COLAMD**, **AMD**) calculent un ordre **réduisant le fill-in** avant de factoriser – ingrédient clé des solveurs SLAM (GTSAM, Ceres, g2o).

Ordering et Fill-in – Visualisation

On **résout** $\Lambda\Delta = \eta$ par élimination dans deux ordres : (ℓ en dernier) vs (ℓ en premier).



ℓ en dernier

ℓ en premier

▶ Étape (0/7)

▶ Résoudre



Chaque étape = une sous-équation ; même Δ , fill-in différent selon l'ordre.

Factorisation de Cholesky

Puisque Λ est **symétrique définie positive**, elle admet une **racine carrée triangulaire** :

$$\Lambda = M^{\top} M$$

M **triangulaire supérieure** (unique). C'est l'analogue matriciel de $\lambda = (\sqrt{\lambda})^2$.

Pourquoi triangulaire ? Un système triangulaire se résout par **substitution** en $O(n^2)$ et bien moins si creux.

Résolution en 2 étapes

Pour résoudre $\Lambda \Delta = M^{\top} M \Delta = A^{\top} b$:

1. **Substitution avant** : $M^{\top} y = A^{\top} b$ (de haut en bas)
2. **Substitution arrière** : $M \Delta = y$ (de bas en haut)

Sparsité préservée

Avec un bon ordering, M reste **creuse** → factorisation quasi-linéaire. C'est ce qui rend le SLAM temps-réel possible.

Factorisation Cholesky

On cherche M **triangulaire supérieure**, avec une diagonale positive, telle que $\Lambda = M^\top M$.

$$\underbrace{\begin{bmatrix} 4 & 2 \\ 2 & 5 \end{bmatrix}}_{\Lambda} = \underbrace{\begin{bmatrix} a & 0 \\ b & c \end{bmatrix}}_{M^\top} \underbrace{\begin{bmatrix} a & b \\ 0 & c \end{bmatrix}}_M = \begin{bmatrix} a^2 & ab \\ ab & b^2 + c^2 \end{bmatrix}$$

- **Entrée (1,1)** : $a^2 = 4 \Rightarrow a = \sqrt{4} = 2$.
- **Entrée (1,2)** : $ab = 2 \Rightarrow b = 2/a = 1$.
- **Entrée (2,2)** : $b^2 + c^2 = 5 \Rightarrow c = \sqrt{5 - b^2} = 2$.

$$M = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$$

On identifie les coefficients, pas seulement une racine entrée par entrée.

Existence et unicité avec cette convention si Λ est symétrique définie positive.

Pour $\Lambda = A^\top A$, cela exige que A soit de rang colonne plein (en SLAM, ça implique avoir un prior).

Cholesky

L'entrée (j, i) de $M^\top M$ est le **produit scalaire des colonnes j et i** de M .

$$\Lambda_{ji} = \sum_{k=1}^j M_{kj} M_{ki} = \underbrace{\sum_{k < j} M_{kj} M_{ki}}_{\text{lignes précédentes : connues}} + M_{jj} M_{ji} \quad (i \geq j)$$

1. Commencer par la diagonale

$$\Lambda_{jj} = \sum_{k < j} M_{kj}^2 + M_{jj}^2$$

$$M_{jj} = \sqrt{\Lambda_{jj} - \sum_{k < j} M_{kj}^2}$$

On retire les carrés déjà connus, puis on choisit la **racine positive**.

2. Compléter la ligne à droite

$$\Lambda_{ji} = \sum_{k < j} M_{kj} M_{ki} + M_{jj} M_{ji}$$

$$M_{ji} = \frac{\Lambda_{ji} - \sum_{k < j} M_{kj} M_{ki}}{M_{jj}} \quad (i > j)$$

On retire les produits déjà connus, puis on **divise par le pivot M_{jj}** .

On répète pour $j = 1, \dots, n$. À la première ligne, les sommes sont vides, donc nulles.

Pourquoi cet ordre ? Chaque calcul n'utilise que les lignes précédentes et le pivot déjà calculé.

Cholesky pas à pas : construire M

Sur notre exemple SLAM 1D : écrire l'égalité → isoler l'inconnue → calculer.

Ligne 1 de M • coefficient 1 / 10

Diagonale → racine positive

Λ donnée, fixe

M à construire

3	-1	0	-1	?	.	.	.
-1	2	-1	0	0	.	.	.
0	-1	2	-1	0	0	.	.
-1	0	-1	2	0	0	0	.

Donnée utilisée

Valeurs réutilisées

Inconnue

Les zéros sous la diagonale sont imposés.

On remplit **une ligne à la fois**, de gauche à droite.

Chaque entrée de Λ est un **produit scalaire de deux colonnes de M**.

1 • Écrire l'entrée (1, 1) du produit $M^T M$

$$\Lambda_{11} = \underbrace{M_{11}^2}_{\text{à déterminer}}$$

2 • Isoler la seule inconnue

$$M_{11} = \sqrt{\Lambda_{11}}$$

Quelle valeur doit remplacer « ? » pour que l'égalité soit vraie ?

Cliquez sur « Calculer » pour la révéler.

← Précédent

Calculer

↺ Recommencer

Affichage arrondi à 3 décimales ; calculs non arrondis.

M est construit : résoudre devient simple

Résolution en 2 étapes

Pour résoudre $\Lambda \Delta = M^\top M \Delta = A^\top b$:

1. **Substitution avant** : $M^\top y = A^\top b$ (de haut en bas)
2. **Substitution arrière** : $M \Delta = y$ (de bas en haut)

1. Substitution avant : $M^\top y = A^\top b$

$$\begin{bmatrix} 1.732 & 0 & 0 & 0 \\ -0.577 & 1.291 & 0 & 0 \\ 0 & -0.775 & 1.183 & 0 \\ -0.577 & -0.258 & -1.014 & 0.756 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} \approx \begin{bmatrix} -7 \\ 0 \\ 1 \\ 6 \end{bmatrix}$$

$$y \approx (-4.041, -1.807, -0.338, 3.780)^\top$$

On calcule y_1 , puis y_2 , y_3 et y_4 .

Aucune matrice à inverser.

2. Substitution arrière : $M \Delta = y$

$$\begin{bmatrix} 1.732 & -0.577 & 0 & -0.577 \\ 0 & 1.291 & -0.775 & -0.258 \\ 0 & 0 & 1.183 & -1.014 \\ 0 & 0 & 0 & 0.756 \end{bmatrix} \begin{bmatrix} \Delta_1 \\ \Delta_2 \\ \Delta_3 \\ \Delta_4 \end{bmatrix} \approx \begin{bmatrix} -4.041 \\ -1.807 \\ -0.338 \\ 3.780 \end{bmatrix}$$

$$\Delta = (0, 2, 4, 5)^\top$$

On calcule Δ_4 , puis Δ_3 , Δ_2 et Δ_1 .

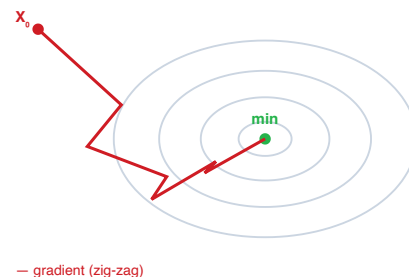
Optimisation Non-Linéaire

Rappel : Optimiser, c'est Descendre vers le Minimum

Minimiser $F(X)$ de façon **itérative** : partir de X_k , choisir une **direction de descente** d (qui fait baisser F), avancer d'un pas α :

$$X_{k+1} = X_k + \alpha \Delta$$

- **Descente de gradient** : $\Delta = -\nabla F$ (plus grande pente). Simple mais **lente**.
- **Newton** : utilise la **courbure** (Hessienne H), $\Delta = -H^{-1}\nabla F$. **Rapide** près du minimum.



Gradient, Jacobien et Hessienne – Moindres Carrés

Coût : $F(X) = \frac{1}{2} \|r(X)\|^2$, résidu $r(X) = h(X) - z$.

Jacobien du résidu : $J = \frac{\partial r}{\partial X}$.

Gradient (règle de la chaîne) :

$$\nabla F = J^\top r$$

Hessienne exacte = $J^\top J + \sum_i r_i \nabla^2 r_i$.

L'algorithme Gauss-Newton, que nous verrons, **néglige** le 2^e terme et approxime:

$$H \approx J^\top J$$

Le pont avec la section précédente

L'étape de Newton $H \Delta = -\nabla F$ devient :

$$J^\top J \Delta = -J^\top r$$

Ce sont **exactement** les équations normales $A^\top A \Delta = A^\top b$ avec

$$A = J \text{ (Jacobien blanchi)}, \quad b = -r$$

$H \approx J^\top J$ ne demande **que le Jacobien** (dérivées 1^{ères}) – aucune dérivée seconde.

Algorithme Gauss-Newton

On enchaîne **linéariser** → **résoudre** → **mettre à jour**, en boucle :

1. **Linéariser** : $r(X_k + \Delta) \approx r(X_k) + J_k \Delta$.
2. **Résoudre** les équations normales (Cholesky) :

$$(J_k^\top J_k) \Delta^* = -J_k^\top r(X_k)$$

3. **Mettre à jour** : $X_{k+1} = X_k \oplus \Delta^*$.

Répéter jusqu'à $\|\Delta^*\| < \epsilon$.

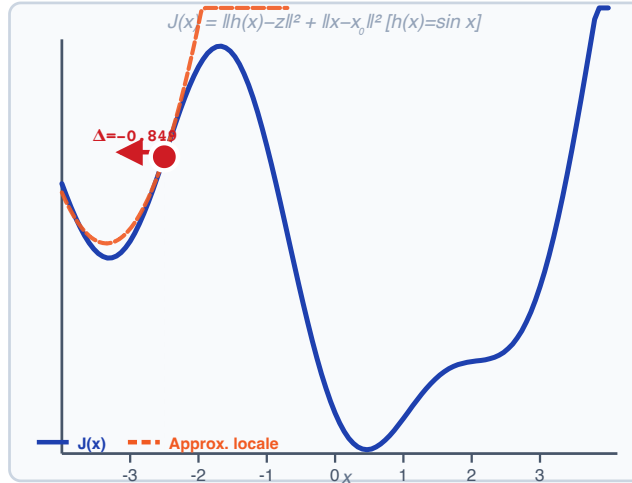
Pseudocode :

```
X ← X₀                                # point de linéarisation initial
Répéter :
  r ← h(X) - z                          # résidu brut
  J ← ∂h/∂X |X                        # Jacobien
  r, J ← W·r, W·J                      # blanchiment : W = Σ(-1/2)
  g ← J⊤·r                            # gradient ∇F (-J⊤r)
  H ← J⊤·J                            # Hessienne approx.
  Δ ← résoudre H·Δ = -g                # Cholesky : H = M⊤M
  X ← X ⊕ Δ                            # composition (≠ addition)
Jusqu'à ||Δ|| < ε (ou ||g|| < ε, i > imax)
```

Convergence

Quadratique près du minimum (très rapide), mais peut **diverger** loin si le pas est trop grand.

Convergence Gauss-Newton – Visualisation



x_0 **-2.5** $k = 0 / 12$

► 1 pas

►► Tout exécuter

↺ Reset

Convergence quadratique près du min, peut diverger loin

État de l'optimisation

Itération $k = 0$

$x_k = -2.5000$ $J = 1.8316$

Pas Gauss-Newton

$r = [h(x) - z, \text{prior}] = [-1.098, -0.791]$

$J = [\partial h / \partial x, \dots] = [-0.801, 0.316]$

$\Delta^* = -(J^T J)^{-1} J^T r = -0.8493$

En cours...

$|\Delta| = 0.8493$

Historique x_k :

-2.500

Levenberg-Marquardt : Robustesse

Problème de Gauss-Newton : loin du minimum, le pas Δ peut être trop grand et diverger.

Solution : amortissement λ sur la diagonale de H :

$$(J^T J + \lambda \text{diag}(J^T J)) \Delta = -J^T r$$

- $\lambda \rightarrow 0$: retrouve le pas de Gauss-Newton.
- λ **grand** : petit pas de gradient préconditionné (voir diapo suivante).

λ est ajusté **dynamiquement** à chaque itération selon un ratio de gain ρ .

Stratégie Trust Region :

Calculer $\rho = \frac{\text{Gain réel}}{\text{Gain prédit}}$.

- $\rho > 0$ accepter (coût baisse) , diminuer λ ($\rightarrow$ GN).
- $\rho \leq 0$ rejeter (coût monte) , augmenter λ ($\rightarrow$ gradient préconditionné).

Algorithme	Vitesse	Robustesse
Gradient Descent	Lent	Haute
Gauss-Newton	Très rapide	Faible
Levenberg-Marquardt	Rapide	Haute

LM : Gauss-Newton, gradient... ou entre les deux ?

À une itération : $H = J^\top J$, $g = J^\top r = \nabla F$ et $D = \text{diag}(H)$.

$$(H + \lambda D) \Delta = -g$$

1. Sans amortissement : $\lambda = 0$

$$H\Delta = -g$$

$$\Delta = -H^{-1}g$$

On retrouve **exactement Gauss-Newton**.

Pour λ petit, le pas est proche du pas GN.

2. Fort amortissement : λD domine H

$$\lambda D\Delta \approx -g$$

$$\Delta \approx -\frac{1}{\lambda}D^{-1}g$$

Petit pas de **gradient préconditionné** : D^{-1} ajuste l'échelle des coordonnées.

Avec $D = I$: $\Delta \approx -\alpha \nabla F$, où $\alpha = 1/\lambda$.

3. Amortissement intermédiaire

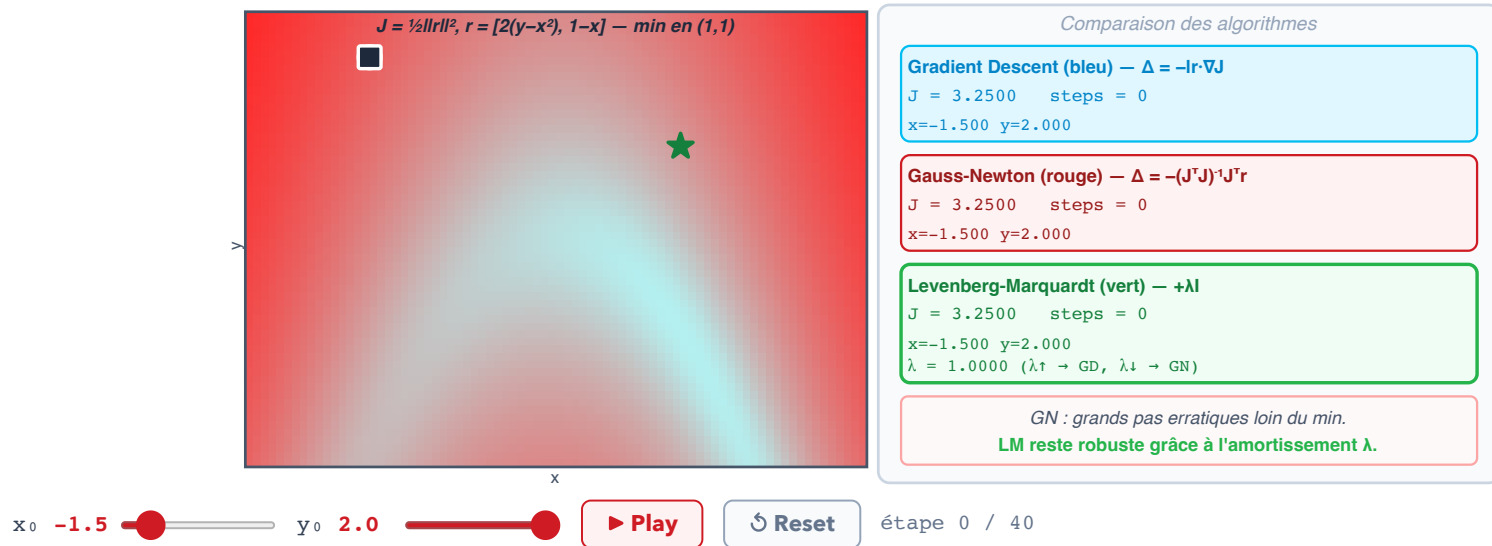
$$H\Delta + \lambda D\Delta = -g$$

$$\Delta = -(H + \lambda D)^{-1}g$$

Les **deux termes comptent** : on conserve l'information de courbure de GN tout en amortissant le pas.

Un compromis, **pas une moyenne pondérée des deux pas**.

LM vs GD vs GN – Trajectoires sur le Paysage



Batch vs Incrémental

Optimisation Batch (Globale) :

À chaque itération, on résout Δ^* pour **toutes** les variables $X_{0:t}$.

- Précis mais coût croissant.
- En SLAM temps réel : le robot opère et accumule des mesures en continue et le coût de calcul croît significativement.

Solution : Inférence Incrémentale

Deux stratégies :

1. **Filtrage (EKF)** : garder seulement la dernière pose, marginaliser l'historique.
2. **Smoothing à fenêtre** : conserver une fenêtre glissante de k dernières poses.

Batch vs Incrémental

Marginalisation ≠ Suppression

Supprimer un nœud = perdre l'information qu'il contient.

Marginaliser un nœud = le retirer du graphe tout en **conservant l'information** via de nouveaux facteurs entre ses voisins.

$$p(y) = \int p(x, y) dx$$

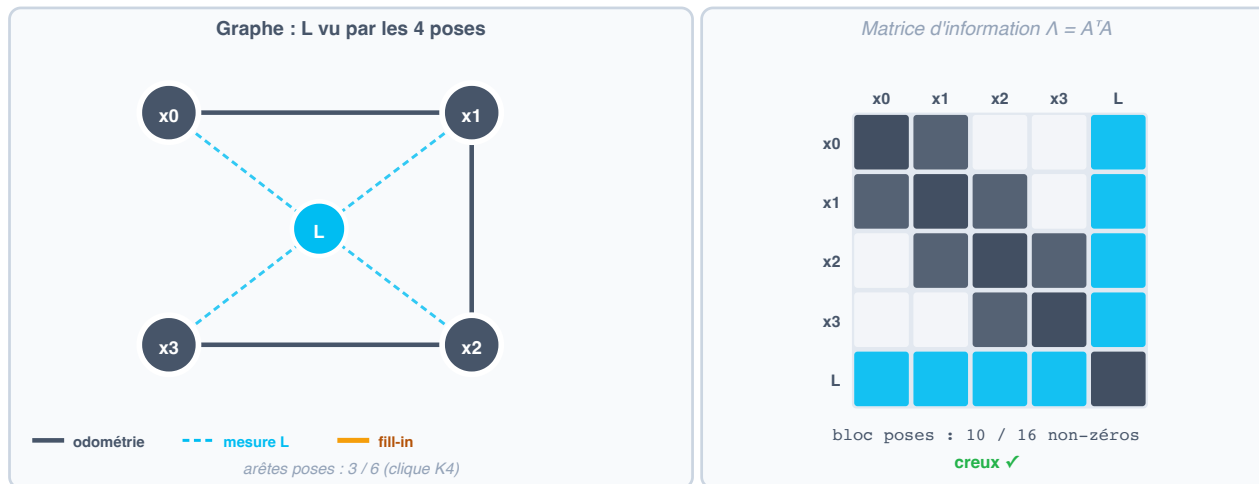
En pratique : marginaliser crée de nouveaux facteurs **denses** entre les variables restantes → la matrice **A** devient moins creuse (fill-in).

EKF

Le filtre de Kalman Étendu (EKF) marginalise **toute** l'historique sauf la pose courante x_t (fenêtre=0). Il ajoute aussi une étape de prédiction.

Permet de bien estimer la position du robot, mais ne préserve pas la carte.

Marginalisation – Avant et Après



► Marginaliser L

↺ Reset

Marginaliser $\neq$ supprimer : retirer L **couple toutes ses voisines** $\rightarrow$ clique dans le graphe, bloc dense dans Λ (fill-in).

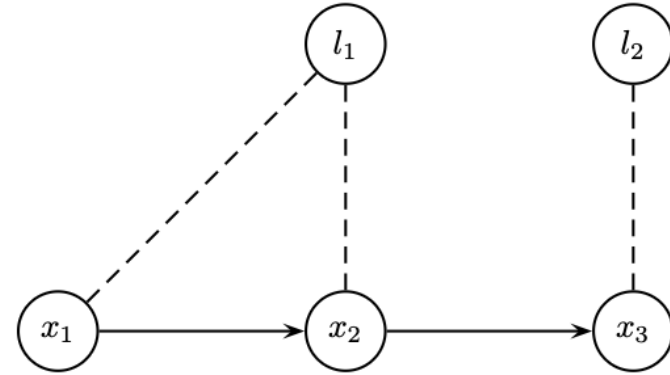
Inférence Incrémentale

En SLAM temps réel, l'information arrive **progressivement** : chaque nouveau pas du robot ajoute une pose et de nouvelles mesures.

Observation clé : la solution précédente Δ_{k-1}^* est un excellent point de départ. On doit **mettre à jour** la factorisation existante plutôt que tout recalculer.

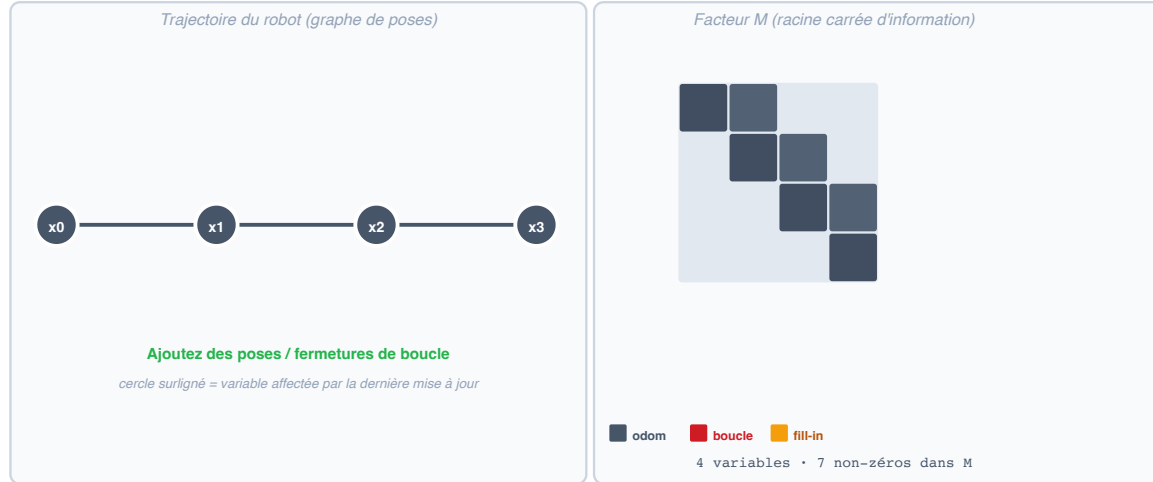
Principe

Ajouter une mesure = ajouter une ligne à la Jacobienne **A**. Comment mettre à jour **M** (facteur de Cholesky) efficacement ?



Carte produite sur le long terme : le problème batch est difficile à maintenir à grande échelle.

Mise à Jour Incrémentale



+ Pose (odométrie)

+ Fermeture de boucle

Reset

Incrémental = *local* : odométrie → 1 bloc ; boucle → bande. Jamais de refactorisation complète $O(n^3)$.

Fixed-Lag Smoothing et Filtrage de Kalman

Désavantage de marginaliser : Ne permet pas de corriger toute la trajectoire.

Compromis : conserver une fenêtre de L poses récentes (lag L) plutôt que toute la trajectoire ou seulement la dernière pose.

Réthode	Avantages	Inconvénients
Batch	Précis et corrige toute la carte	coûteux en calcul et en mémoire
Fixed-Lag	Rapide	Trajectoire (et carte) non corrigée
EKF	Rapide	Ne maintient pas de carte corrigée, nécessite de modéliser la dynamique du robot, peut diverger.

Groupes de Lie

Problématique – $SO(3)$ n'est pas un espace euclidien

En optimisation, la mise à jour classique est :

$$X_{k+1} = X_k + \Delta$$

Problème : Pour les rotations $R \in SO(3)$, cette addition détruit les propriétés.

$$R_{\text{new}} = R + \Delta \notin SO(3)$$

car R_{new} ne vérifie plus :

- **Orthogonalité :** $R^T R = I_3$
- **Dextrogyre :** $\det(R) = +1$

Conséquence

On ne peut pas utiliser un solveur linéaire directement sur les rotations. Il faut un outil mathématique adapté à la **géométrie** de l'espace des rotations.

L'espace $SO(3)$ est une variété courbe

$SO(3)$ est une **variété (manifold)** de dimension 3 plongée dans $\mathbb{R}^{3 \times 3}$.

Il ressemble localement à $\mathbb{R}^3$ (on peut définir un espace tangent), mais globalement c'est une sphère courbe, pas un espace vectoriel plat.

Groupes et Groupes de Lie

Un **groupe** G est un ensemble avec une opération binaire $\otimes$ satisfaisant :

- **Fermeture** : $\forall A, B \in G, A \otimes B \in G$
- **Associativité** : $(A \otimes B) \otimes C = A \otimes (B \otimes C)$
- **Élément Identité** : $\exists I$ tel que $A \otimes I = I \otimes A = A$
- **Inverse** : $\forall A, \exists A^{-1}$ tel que $A \otimes A^{-1} = I$

Un **groupe de Lie** est un groupe dont les opérations de groupe sont différentiables.

Groupes importants en robotique :

Groupe	Description	Dim.
$SO(2)$	Rotations 2D	1
$SO(3)$	Rotations 3D	3
$SE(2)$	Poses 2D (R, t)	3
$SE(3)$	Poses 3D (R, t)	6

Pourquoi les groupes de Lie ?

Ils permettent un **traitement unifié** des rotations et poses, définissent formellement la notion de **distance** entre poses, et permettent l'**optimisation rigoureuse** sur la variété (on-manifold optimization).

Distances sur $SO(3)$

Soit $R_A, R_B \in SO(3)$.

1. Distance Géodésique (angulaire)

L'angle minimal pour aligner les deux repères :

$$d_\theta(R_A, R_B) = \arccos\left(\frac{\text{tr}(R_A^T R_B) - 1}{2}\right)$$

2. Distance Chordale (Frobenius)

Distance euclidienne dans l'espace des matrices :

$$d_c(R_A, R_B) = \|R_A - R_B\|_F$$

où

$$\|\mathbf{M}\|_F^2 = \sum_{i,j} M_{ij}^2 = \text{tr}(\mathbf{M}^\top \mathbf{M})$$

Analogie sur $SO(2)$ – le cercle unitaire :

- L'**arc** (chemin sur le cercle) = distance géodésique.
- La **corde** (ligne droite) = distance chordale.

Distances sur $SE(3)$

Soit $T_A = (R_A, t_A), T_B = (R_B, t_B) \in SE(3)$.

1. Double Distance Géodésique

Découple rotation et translation :

$$d_{dg}(T_A, T_B) = \sqrt{d_\theta(R_A, R_B)^2 + \|t_B - t_A\|^2}$$

Problème : unités incompatibles (radians vs mètres).

2. Distance Chordale $SE(3)$

$$\begin{aligned} d_c(T_A, T_B) &= \|T_A - T_B\|_F \\ &= \sqrt{d_c(R_A, R_B)^2 + \|t_B - t_A\|^2} \end{aligned}$$

Non bi-invariance de $SE(3)$

Contrairement à $SO(3)$, les distances sur $SE(3)$ ne sont pas **bi-invariantes** :

$$d(T_A, T_B) = d(T_C T_A, T_C T_B) \neq d(T_A T_C, T_B T_C)$$

Cela signifie que la distance entre deux poses dépend du repère de référence choisi. Il n'existe pas de métrique bi-invariante "naturelle" sur $SE(3)$.

Algèbres de Lie : $\mathfrak{so}(3)$ et $\mathfrak{se}(3)$

L'algèbre de Lie est l'espace tangent à l'identité du groupe de Lie.

$\mathfrak{so}(3)$ – Algèbre de $SO(3)$

Ensemble des matrices skew-symmetric 3×3 :

$$\phi^\wedge = [\phi]_\times = \begin{bmatrix} 0 & -\phi_3 & \phi_2 \\ \phi_3 & 0 & -\phi_1 \\ -\phi_2 & \phi_1 & 0 \end{bmatrix}$$

Opérateurs **hat** $(\cdot)^\wedge$ (vecteur $\rightarrow$ matrice) et **vee** $(\cdot)^\vee$ (matrice $\rightarrow$ vecteur).

$\mathfrak{se}(3)$ – Algèbre de $SE(3)$

Vecteur de perturbation $\xi = [\rho^T, \phi^T]^T \in \mathbb{R}^6$:

$$\xi^\wedge = \begin{bmatrix} \phi^\wedge & \rho \\ 0^T & 0 \end{bmatrix} \in \mathbb{R}^{4 \times 4}$$

- $\rho \in \mathbb{R}^3$: vitesse linéaire (perturbation de t).
- $\phi \in \mathbb{R}^3$: vitesse angulaire (perturbation de R).

Pourquoi l'algèbre de Lie ?

L'algèbre de Lie est un **espace vectoriel** (plat, linéaire). C'est là qu'on peut faire des additions et calculer des gradients, puis on remonte sur la variété via la carte exponentielle.

Calcul sur les Groupes de Lie

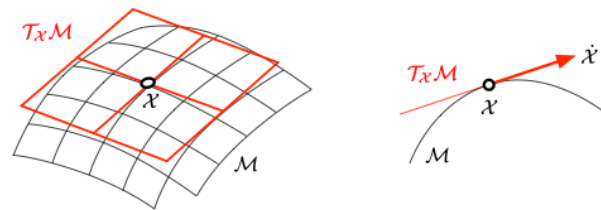
Variétés et Espace Tangent

Définition : Une variété de dimension d est un espace topologique qui ressemble **localement** à $\mathbb{R}^d$.

Espace tangent $T_X \mathcal{M}$: espace vectoriel linéaire tangent à la variété au point X .

- Permet de définir des **directions de déplacement** locales.
- C'est l'espace dans lequel on calcule les gradients et Jacobiens.

Un **groupe de Lie** est une variété lisse dont les opérations de groupe sont différentiables.



L'espace tangent en un point de la variété

Intuition

Imaginez la surface d'une sphère. En chaque point, un plan tangent "colle" localement à la sphère. On optimise dans ce plan, puis on revient sur la sphère.

Cartes Exponentielles et Logarithmiques

Ces applications relient l'**algèbre de Lie** (espace tangent, **linéaire**) au **groupe de Lie** (variété, **non-linéaire**) :

Carte Exponentielle : Algèbre → Groupe

$$G = \text{Exp}(\phi) = \exp(\phi^\wedge)$$

Pour $SO(3)$ – formule de Rodrigues :

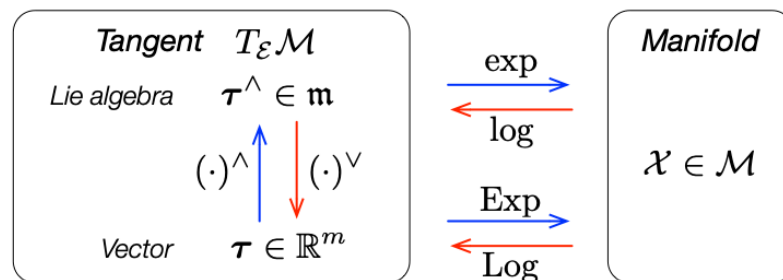
$$R = I + \sin \theta u^\wedge + (1 - \cos \theta) (u^\wedge)^2$$

avec $\phi = \theta u$ (angle $\theta = \|\phi\|$, axe $u = \phi / \|\phi\|$).

Carte Logarithmique : Groupe → Algèbre

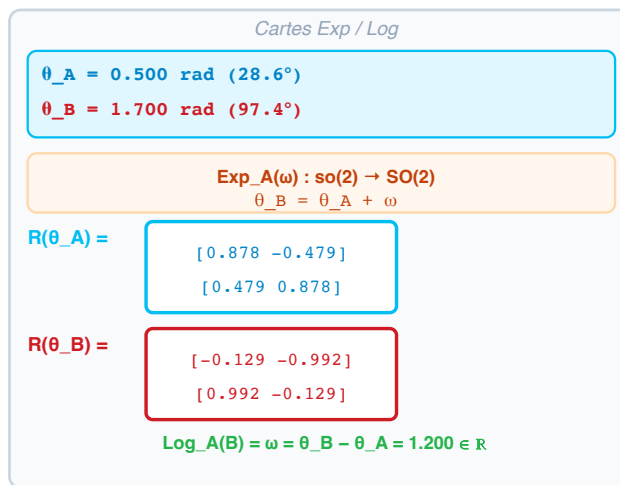
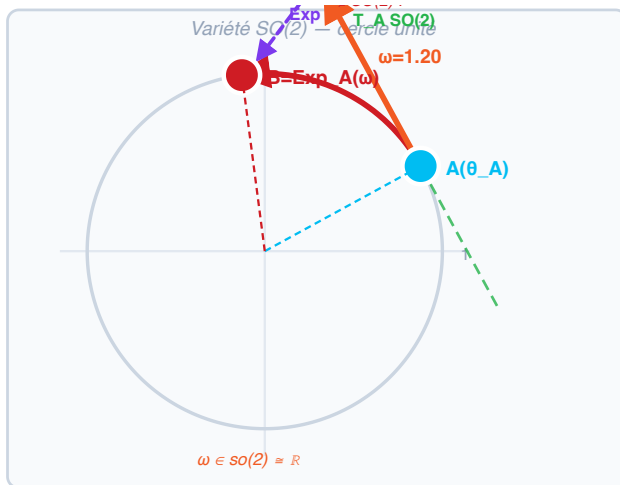
$$\phi = \text{Log}(G) = \log(G)^\vee$$

Pour $SO(3)$: $\theta = \arccos\left(\frac{\text{tr}(R)-1}{2}\right)$



Résumé des mappings Exp et Log

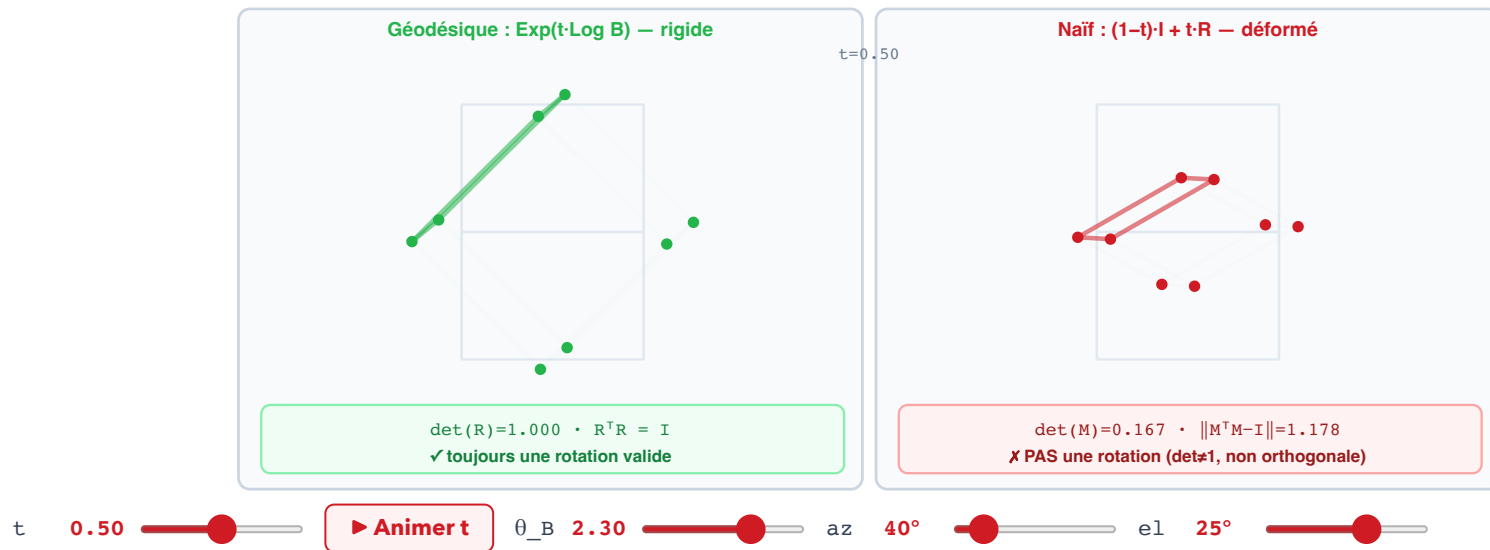
SO(2) – Variété, Tangente et Carte Exponentielle



θ_A 0.50 ω 1.20

L'addition naïve (le long de la tangente) quitte $SO(2)$ · **Exp** rétracte sur la variété $\rightarrow B$

SO(3) – Addition Géodésique vs Naïf



On ne peut pas additionner des rotations → il faut Exp/Log (géodésique).

Opérateurs $\oplus$ et $\ominus$ – Calcul sur la Variété

Opérateur Plus $\oplus$: perturber un élément du groupe :

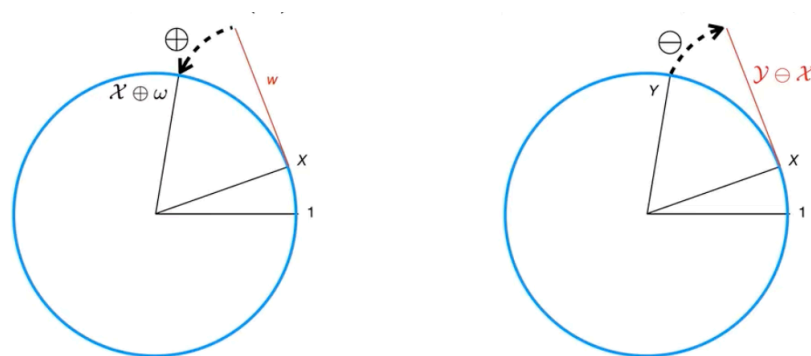
$$X \oplus \omega \doteq X \cdot \text{Exp}(\omega)$$

- $X \in G$: état courant (sur la variété).
- $\omega \in \mathbb{R}^n$: perturbation dans l'espace tangent (vecteur).
- $X \oplus \omega \in G$: nouvel état (sur la variété).

Opérateur Moins $\ominus$: différence entre éléments :

$$X \ominus Y \doteq \text{Log}(Y^{-1} \cdot X)$$

Donne la perturbation dans l'espace tangent qui transforme Y en X .



Opérateurs $\oplus$ et $\ominus$ sur la variété

Lift – Solve – Retract

Le paradigme d'optimisation sur les variétés, utilisé avec un solveur comme Gauss-Newton et L-M.

① Lift (Soulever)

On **transfère** le problème depuis la variété courbe vers l'**espace tangent linéaire** local en X_k .

On calcule les résidus $r_i = h_i(X_k) - z_i$ et les Jacobiens $J_i = \frac{\partial h_i}{\partial X} \Big|_{X_k}$.

② Solve (Résoudre)

Dans l'espace tangent (**vectorel, plat**), on calcule le pas optimal par les équations normales :

$$(J^T J) \Delta^* = -J^T r$$

Résolution par Cholesky, exploite la **sparsité** du graphe de facteurs.

③ Retract (Rétracter)

On **ramène** l'incrément Δ^* sur la variété via l'opérateur $\oplus$:

$$X_{k+1} = X_k \oplus \Delta^*$$

Le X_{k+1} résultant est un élément valide du groupe (rotation, pose...).

Itérations jusqu'à convergence

Lift – Solve – Retract sur une courbe 3D

Le problème

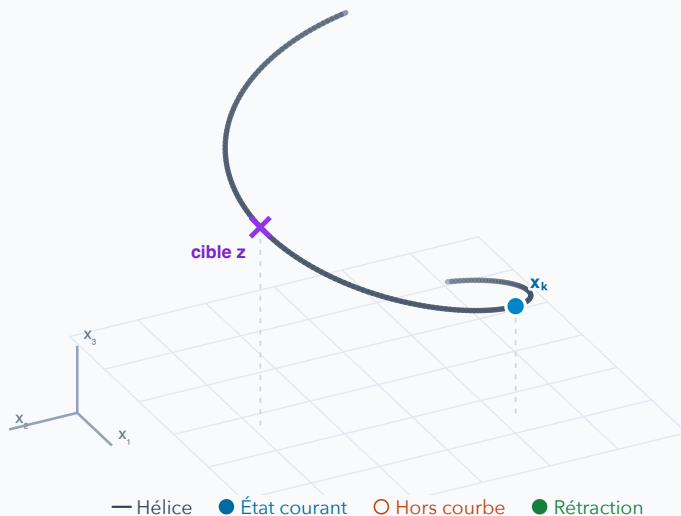
① Lift · linéariser

② Solve · calculer

③ Retract · revenir

Une courbe 1D dans $\mathbb{R}^3$

Glisser pour tourner · touches fléchées



Itération 0

$F(x_k) = 1.6611$

Un état contraint à rester sur l'hélice

$$q(t) = \begin{bmatrix} \cos t \\ \sin t \\ 0.35t \end{bmatrix}, \quad x_k = q(t_k)$$

La position x a **3 coordonnées**, mais un seul degré de liberté : le paramètre t .

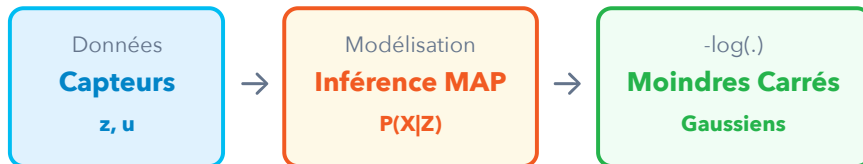
$$\min_t F(t) = \frac{1}{2} \|q(t) - z\|^2$$

On cherche à rejoindre la cible violette $z = q(1,35)$, située sur la courbe.

Une droite tangente suffit : la dimension de l'espace tangent est **1**, pas 3.

Résumé – Le Flow d'Estimation d'État Complet

Pipeline d'Estimation d'État : de Bout en Bout



Résolution du problème de moindres carrés :



Les étapes en rouges bouclent : Lift $\rightarrow$ Solve $\rightarrow$ Retract... jusqu'à convergence $\|\Delta^*\| < \epsilon$